```
affine.for %bk = 0 to 16 {
  // Load from "SM(Q)" to "Reg(Q)"
  affine.for %i = 0 to 2 {
    %0 = affine.load %smemQ[%k + %bk, (%tid floordiv 32) * 8 + ((%tid mod 32) floordiv 8) * 2 + %i] : memref<128x32xf32>
    affine.store %0, %regQ[%i] : memref<2xf32>
  }
  // Load from "SM(K)" to "Reg(K)"
  affine.for %j = 0 to 4 {
    %0 = affine.load %smemK[%bk, (%tid mod 8) * 4 + %j] : memref<16x32xf32>
    affine.store %0, %regK[%j] : memref<4xf32>
  }
  // Exterior product
  affine.for %i = 0 to 2 {
    affine.for %j = 0 to 4 {
      %0 = affine.load %regQ[%i] : memref<2xf32>
      %1 = affine.load %regK[%j] : memref<4xf32>
      %2 = affine.load %acc_s[%i, %j] : memref<2x4xf32>
      %3 = arith.mulf %0, %1 : f32
      %4 = arith.addf %2, %3 : f32
      affine.store %4, %acc_s[%i, %j] : memref<2x4xf32>
    }
  }
}
```